# EarningsLens

## User Guide

An analyst-style read of two earnings calls,
powered by local Hugging Face models.

**Version 1.1**
Document type: End-user guide
Audience: Analysts, students, and reviewers

**Team**
Augustin BAUDANT • Yuzhi LUO • Ludovic SAINT-YVES
Octave SUPRANO • Jin Wei ZHENG YANG

# Welcome to EarningsLens

EarningsLens reads two consecutive earnings call transcripts from the same company and produces a concise, analyst-style brief. It runs five independent analyses — sentiment, hedging, topic drift, risk vocabulary, and Q&A evasion — then synthesises the structured outputs into a short narrative.

Everything runs locally on your machine using open-source Hugging Face models. Nothing is sent to a remote LLM. The output is an analytical aid: it is **not** investment advice.

## What you will get

- A headline **GREEN / AMBER / RED** signal summarising the quarter-over-quarter read.
- Five diagnostic tabs you can drill into independently.
- A short, written analyst brief that grounds itself in the structured outputs.
- Provenance for every analysis so you know whether the score came from the local LLM or a deterministic backup pass.
- Hard errors are surfaced directly — if an analysis fails, the app halts and shows the exception rather than silently substituting placeholder numbers.

## The headline signal

| | |
|---|---|
| **GREEN** | No major issues flagged across sentiment, hedging, topics, and Q&A responsiveness. |
| **AMBER** | One analytical dimension moved enough to warrant monitoring next quarter. |
| **RED** | Multiple independent stress signals appeared at once — read the transcripts closely. |

Think of the signal as a triage layer. GREEN does not mean the quarter was strong — it means nothing in the language warranted special attention. RED does not mean the company is in trouble — it means several independent linguistic stress markers moved together and a human should look closely.

# Getting started

## 1. Requirements

- Python 3.11
- Around 1 GB of disk for the first run (FinBERT ~440 MB, MiniLM ~80 MB, the local instruct model ~1 GB).
- 8 GB of RAM is enough thanks to the lightweight default model (*SmolLM2 1.7B*).
- An Alpha Vantage API key, if you want to load real tickers directly inside the app.

## 2. Configure your .env

Copy **.env.example** to **.env** at the project root and fill in the two variables below. The file is read at startup; restart the app after editing it.

| Variable | Req? | What it does | Default |
|---|---|---|---|
| `MODEL` | Opt | Hugging Face model id for the local instruct LLM used by topic extraction, Q&A evasion scoring, and brief synthesis. | SmolLM2-1.7B-Instruct |
| `ALPHA_VANTAGE_API_KEY` | If search | Free Alpha Vantage key used by **Search company** mode to fetch real ticker transcripts. Not needed for **Use sample** mode. | — |
| `EVASION_BATCH_SIZE` | Opt | Number of Q&A prompts batched into a single LLM forward pass when scoring evasion. Higher values are faster but use more memory; reduce on tight RAM. | 3 |
| `LOW_MEMORY_MODE` | Opt | When enabled, runs the heaviest model-backed analyses in phases to reduce peak RAM. Set to 0 on larger machines for the faster parallel path. | 1 |

```
MODEL=HuggingFaceTB/SmolLM2-1.7B-Instruct
ALPHA_VANTAGE_API_KEY=your_alpha_vantage_key
```

Grab a free Alpha Vantage key at *alphavantage.co/support/#api-key*. Without it you can still use **Use sample** mode — only ticker search needs the key.

## 3. Install (one-click)

The fastest path for a graded reviewer is the one-click installer:

- **macOS** — double-click **setup.command** in Finder.
- **Windows** — double-click **setup.bat** in File Explorer.

Both scripts (1) verify that Python 3.11 is present, (2) create the local **.venv**, (3) install every pinned dependency from **requirements.txt**, (4) copy **.env.example** to **.env**, and (5) pre-download the three Hugging Face models (~1.5 GB total) into the on-disk cache. If Python 3.11 is missing, the script stops and tells you exactly where to download it from *python.org* — every other step is automated. Re-running the script is safe; existing components are reused, not reinstalled.

## 3b. Manual install (if you prefer)

```
python3.11 -m venv .venv
source .venv/bin/activate   # Windows: .venv\Scripts\activate
pip install -r requirements.txt
cp .env.example .env
python scripts/prefetch_models.py
streamlit run app.py
```

## 4. Launching the app

After setup, start the app without typing any commands: double-click **launch.command** from Finder on macOS, or **launch.bat** from File Explorer on Windows. Both scripts activate the local virtualenv, verify dependencies, and launch Streamlit. Subsequent launches start in seconds thanks to the on-disk model cache.

## 5. Optional health checks

```
./.venv/bin/pytest -q
./.venv/bin/python scripts/health_check.py
```

These verify that parsing, scoring, and model loading behave as expected before you trust any output.

# Using the app

## Input modes

EarningsLens supports two input modes, chosen from the sidebar:

- **Search company.** Type an exact ticker, select a match, load the available quarters, and pick the two you want to compare. Transcripts are fetched via Alpha Vantage.
- **Use sample.** Run on built-in demo pairs — Microsoft and Goldman Sachs Q-over-Q comparisons. Perfect for trying the tool offline or for grading without API calls.

## Sidebar controls

- **Mode** — choose **Search company** to fetch real transcripts by ticker, or **Use sample** for built-in demo pairs.
- **Input selection** — use sidebar dropdowns to pick or search companies and compare quarters.
- **Show diagnostics** — toggle to surface parsed speaker roles, turn segmentation, evasion confidence scores, and analysis provenance.
- **Configuration** — thresholds are set in **.env**, not the sidebar. Keep **LOW_MEMORY_MODE=1** for lower peak RAM, or set it to 0 for faster parallel execution.

## The five analyses

| Tab | What it measures | What to look at |
|---|---|---|
| **Sentiment** | FinBERT tone of prepared remarks vs. tone during Q&A. | A widening negative gap means the unscripted portion was darker than the script. |
| **Hedging** | Use of uncertainty words (may, could, pressure, headwind…) normalised per turn. | A sharp quarter-over-quarter rise often signals weaker conviction. |
| **Topics** | Semantic similarity of prepared remarks via MiniLM + themes extracted by the local LLM. | New, risk-heavy themes and dropped themes are highlighted. |
| **Risk vocab** | Curated finance vocabulary (inflation, liquidity, competition, backlog…). | Biggest movers surface first — useful for narrative shifts. |
| **Q&A Evasion** | LLM-as-judge rubric scoring whether each answer addressed its question. | Low scores flag exchanges that a human should re-read, not proof of misconduct. |

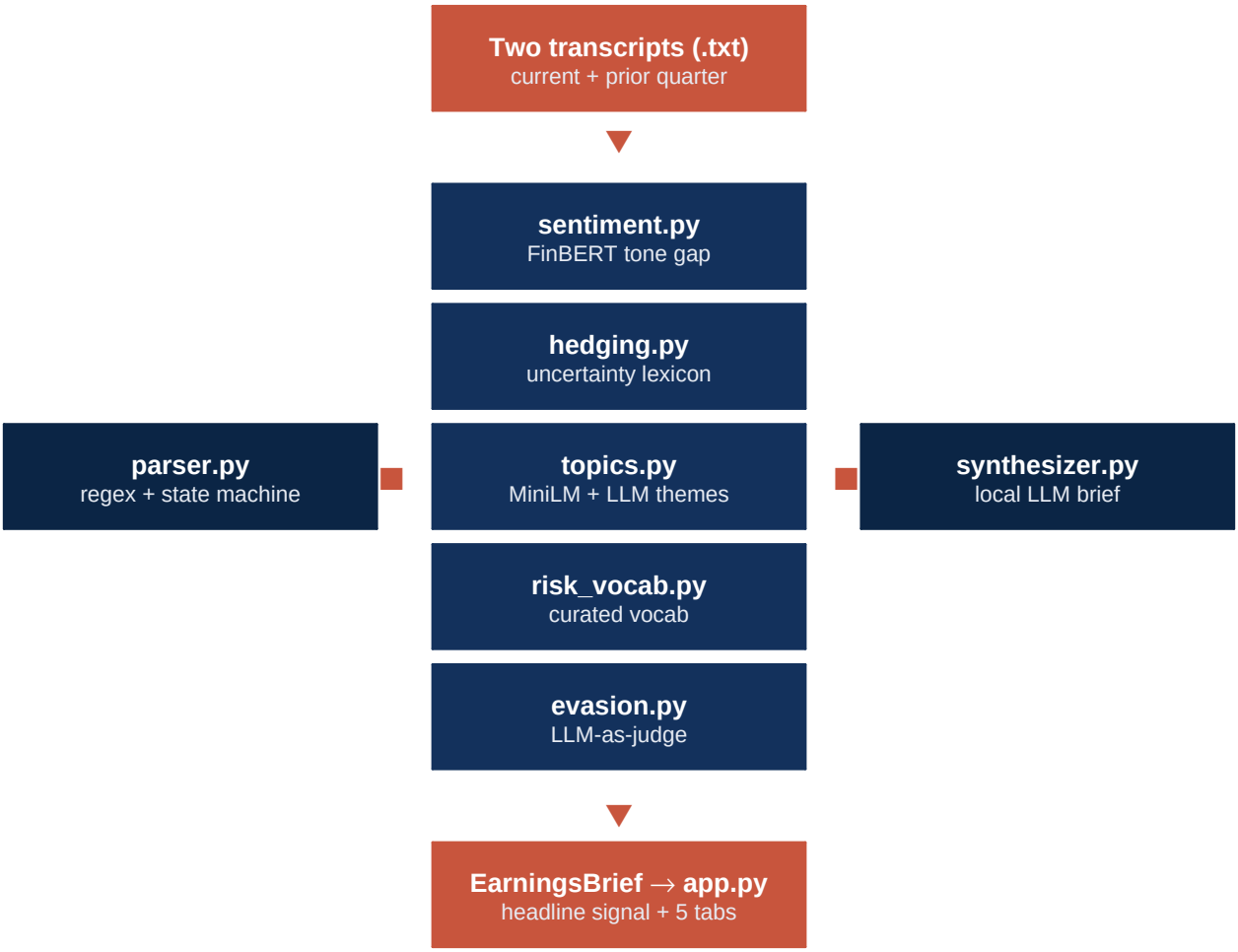
## Reading the analyst brief

The brief at the top of the page is generated by the local instruct model from the structured outputs above — not from the raw transcripts. That keeps the synthesis anchored to explicit evidence and reduces the risk of hallucinated quotes. When the LLM returns an unusable response for a specific Q&A pair or theme list, a deterministic backup pass (heuristic scoring, n-gram theme extraction, rule-based summary) takes over for just that piece and the provenance badge marks it accordingly.

# Under the hood

EarningsLens is intentionally layered so each signal can be inspected and trusted independently. Rule-based components are transparent; ML components are scoped narrowly.

## Pipeline




## Why this mix of methods

- **Parsing** is regex + a small state machine. The transcript format is repetitive enough that rules are more reliable than an LLM.
- **Sentiment** uses FinBERT because general-purpose sentiment misreads financial language (e.g. *“flat margins”*).
- **Hedging & risk vocab** are explicit lexicons. The signal is fully auditable and runs instantly on CPU.
- **Topics** mix MiniLM embeddings (broad drift) with the local LLM (human-readable themes).
- **Evasion** uses an LLM-as-judge rubric on a local model so no transcript ever leaves your machine.

## Local model

The default local instruct model is **HuggingFaceTB/SmolLM2-1.7B-Instruct**. It is lightweight enough to run on an 8 GB Mac while still capable on short structured tasks: theme extraction, Q&A rubric scoring, and brief synthesis. You can swap it by editing the model id in **.env**.

# Troubleshooting & tips

### First run is slow

That is expected — three models are being downloaded and cached. Subsequent runs reuse the cache and are much faster.

### The .models folder is huge

EarningsLens only uses the PyTorch weights of each model, but a fresh Hugging Face download also ships ONNX, TensorFlow, Flax, OpenVINO and Rust variants we never load. Run **python scripts/clean_model_cache.py** to delete those — typically reclaims ~12 GB and leaves a ~1.3 GB on-disk cache that the app still loads from without issue. The updated **prefetch_models.py** avoids re-downloading the unwanted variants on future installs.

### A score shows a heuristic or extract badge

The local LLM returned an unparseable response for that item, so a deterministic backup pass took over (heuristic Q&A scoring, n-gram theme extraction, or rule-based summary). The score is still valid but is built from explicit rules rather than the model — treat it as lower-confidence and re-read the underlying turn if it drives a flag.

### The app stops with a red error box

An analysis raised an exception. EarningsLens no longer silently substitutes placeholder numbers — the failing stage and its exception are shown verbatim and the run is aborted so you don't read a brief built from missing inputs. Common causes: a transcript that is too short, a model file that did not finish downloading, or an out-of-memory error on the local LLM. Re-run after addressing the cause.

### Q&A pairs look mismatched

The parser merges consecutive turns from the same speaker and strips common openers (*“A separate question,” “One quick one,”*) and conversational fillers (*“Yeah, sure, thanks for the question…”*) before scoring. Pairs that still look ambiguous — moderator interjections, paste artefacts, role uncertainty — are downweighted by the parser-confidence threshold (0.55) and excluded from flags. Enable **Show diagnostics** to inspect per-pair confidence.

### Alpha Vantage rate limits

The free tier is limited. If a fetch fails, wait a minute and retry, or fall back to the sample mode while you iterate.

# Supporting documents

EarningsLens ships with a small set of companion documents so reviewers can trace any claim in the app back to its source. They live next to the code in the repository.

| File | What's inside |
|---|---|
| `README.md` | Project overview, setup, run instructions, evaluation rationale (threshold tuning), and full credits for packages, datasets and methods. |
| `docs/EarningsLens_User_Guide.pdf` | This document. End-user guide with screenshots, sidebar walk-through and troubleshooting. |
| `docs/architecture.md` | Pipeline architecture, module responsibilities, and data-flow diagram. |
| `docs/course-concepts.md` | Row-by-row mapping between the concepts taught in *Introduction to AI for Business* and the files where each concept is applied. |
| `PROMPTS.md` | Log of the prompts that were issued to AI coding assistants (Claude Code, ChatGPT, GitHub Copilot) during development, with a description of which code each set of prompts produced. |
| `data/samples/SOURCES.md` | Attribution, call dates and licensing notes for the four bundled earnings-call transcripts. |
| `tests/` | 25 **pytest** regression tests covering the parser, the evasion rubric, score-adjustment heuristics, the question splitter and the answer-filler stripper. |

## How thresholds were set

The five flag thresholds — sentiment gap, hedging delta, topic similarity, evasion responsiveness and parser confidence floor — were tuned on the four bundled sample transcripts plus a handful of additional public calls that are not committed to the repository. The rationale for each value is documented in the **Evaluation** section of **README.md**. The numbers live in **earningslens/config.py** and can be overridden via environment variables.

## Use of AI coding assistants

Per the assignment brief, the dev-time use of AI coding tools (Claude Code, ChatGPT, GitHub Copilot) is logged in **PROMPTS.md** at the repository root. That file lists the prompts that were used to scaffold the parser, design the Q&A rubric, audit performance, and draft the documentation. All generated code was reviewed and tested by a human team member before being committed.

## Sample transcript attribution

The Goldman Sachs and Microsoft transcripts under **data/samples/** are publicly available earnings calls reproduced here for non-commercial educational use. Full attribution, call dates, and a citation-ready format are documented in **data/samples/SOURCES.md**.

## A note on responsible use

EarningsLens surfaces *linguistic* signals — sentiment shifts, hedging creep, topic drift, low-responsiveness Q&A exchanges. None of these prove anything on their own. They are starting points for human reading, not conclusions. The banner at the top of the app makes the same point: the output is an analytical aid, not investment advice.